

Homework 5

by Petros Efraim Koukios, and Alex Rystrom
Group 68

February 24, 2026

Problem 5: The Stable Marriage problem (40 points)

In this assignment, the Stable Marriage problem was solved using a distributed approach with the Message Passing Interface (MPI). The algorithm ensures that in a population of n men and n women, everyone is matched such that no two people of opposite sex would both rather have each other than their current partners. The implementation utilises a SPMD (Single Program, Multiple Data) model where each participant acts as an individual MPI process.

Distributed Logic and Process Roles

The solution splits the total number of processes $2n$ into two distinct groups based on their MPI rank. Ranks $[0, n - 1]$ represent men, while ranks $[n, 2n - 1]$ represent women. This separation allows for distinct behaviour while maintaining a shared communication.

- **Men (Proposers):** Active processes that iterate through a shuffled preference list, sending `TAG_PROPOSE` messages to women until they are engaged and the system stabilises.
- **Women (Reviewers):** Passive processes that respond to proposals. They maintain a rank-based lookup table `build_rank_array` to compare current partners against new proposers in constant time $O(1)$.

Communication Protocol and State Management

The core of the algorithm is implemented through asynchronous message passing. Unlike a centralised monitor, synchronisation is achieved through specific MPI tags that dictate the state of the participants.

MPI Tag	Description
<code>TAG_PROPOSE</code>	Sent by a man to his next preferred woman.
<code>TAG_ACCEPT</code>	Sent by a woman to confirm an engagement.
<code>TAG_REJECT</code>	Sent by a woman if the proposer is less preferred than her current partner.
<code>TAG_DUMP</code>	Sent to a previously engaged man when a better offer arrives.
<code>TAG_TERMINATE</code>	Broadcast to stop all processes once stability is reached.

Table 1: Communication protocol used for coordination.

Termination via the Checklist Pattern

A significant challenge in a distributed Stable Marriage implementation is detecting when the system has reached a stable state (i.e., all participants are matched). This code implements a Checklist Passing mechanism instead of using an extra counter process:

1. The first woman (rank n) starts with the `TAG_CHECKLIST`.

2. Once she is engaged, she passes the checklist to the next woman (rank $rank + 1$).
3. The checklist only moves forward when a woman is currently matched.
4. When the last woman (rank $2n - 1$) receives the checklist and is engaged, it serves as proof that every woman is matched, and thus the system is stable.

Implementation Details

The implementation uses a shuffle to generate unique, random preference lists for every process. To prevent deadlocks during the termination phase, the last woman performs a broadcast of TAG_TERMINATE to all $2n - 1$ other processes, ensuring a clean exit from the MPI_Recv loops.

In conclusion, the MPI-based solution effectively parallelises the proposal process. By using a token-passing "checklist" among the women, the distributed system can reach a consensus on termination without the need for a centralised "master" process monitoring every engagement state.

```
Trace: Man 1 proposes to Woman 0
Trace: Man 3 proposes to Woman 0
Trace: Man 2 proposes to Woman 3
Trace: Man 0 proposes to Woman 2
Trace: Man 1 is engaged to Woman 0
Trace: Man 2 is engaged to Woman 3
Trace: Man 0 is engaged to Woman 2
Trace: Man 3 is rejected by Woman 0
Trace: Man 3 proposes to Woman 1
Trace: Man 3 is engaged to Woman 1

--- System Stable! Broadcasting Termination ---

Result: Man 2 final partner: Woman 3
Result: Man 0 final partner: Woman 2
Result: Man 1 final partner: Woman 0
Result: Man 3 final partner: Woman 1
Result: Woman 0 final partner: Man 1
Result: Woman 3 final partner: Man 2
Result: Woman 1 final partner: Man 3
Result: Woman 2 final partner: Man 0
```

Figure 1: Example output of program run with 8 people. (4 men, 4 women)